

Real Time Cluster Path Tracing

Feng Xie
Facebook Reality Labs
USA
feng.xie@fb.com

Petro Mishchuk
Apex Systems
USA
pmishchuk@fb.com

Warren Hunt
Facebook Reality Labs
USA
whunt@fb.com



Figure 1: Trudy as Black Swan rendered with our cluster based photorealistic rendering system.

CCS CONCEPTS

• Computing methodologies → Rendering; Raytracing; Distributed Algorithms; Parallel Algorithms.

KEYWORDS

Real Time Path Tracing, Distributed Rendering, Scalability and Performance, Real Time Global Illumination

ACM Reference Format:

Feng Xie, Petro Mishchuk, and Warren Hunt. 2021. Real Time Cluster Path Tracing. In *SIGGRAPH Asia 2021 Technical Communications (SA '21 Technical Communications)*, December 14–17, 2021, Tokyo, Japan. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3478512.3488605>

1 INTRODUCTION

Photorealistic rendering effects are common in films, but most real time graphics today still rely on scanline based multi-pass rendering to deliver rich visual experiences.

In this paper, we present the architecture and implementation of the first production quality real-time cluster path tracing renderer. We build our cluster path tracing system using the open source Blender and its GPU accelerated production quality renderer Cycles [Blender 2021]. Our system's rendering performance and quality scales linearly with the number of RTX cluster nodes used. It is able to generate and deliver path traced images with global illumination

effects to remote light-weight client systems at 15 – 30 frames per second for a wide variety of Blender scenes including virtual objects and animated digital human characters.

2 RELATED WORK

The path tracing revolution in CG production started around 2007. With the release of the film *Cloudy with a Chance of Meatballs*, Sony rocked production rendering by using the brute force path tracer Arnold [Kulla et al. 2018]. Since then most major CG film productions have moved onto path traced rendering. In 2018, TOG published a special issue on production path tracing renderers including Maya [Georgiev et al. 2018] and Sony Arnold [Kulla et al. 2018], Weta's Manuka [Fascione et al. 2018], Disney's Hyperion [Burley et al. 2018] and Pixar's Renderman [Christensen et al. 2018]. These papers present the benefits of physically based rendering w.r.t. artist workflow, image quality consistency and predictability. Path tracing isn't just able to deliver physically based photorealism, but can also be used to create highly stylized films like Sony's *Spiderman in the Spider-verse* and DreamWorks' furry *Trolls*.

For real time PC and console games, the launch of Nvidia's RTX GPUs and Intel's Xe Graphics, along with the support for ray tracing APIs in Direct X and Vulkan translated to broader adoption of ray tracing based reflections and shadows. However, most games today still use scanline based rendering system as a foundation, with effects like global illumination, subsurface scattering, area lights, environment lighting supported using a multi-pass or light baking approach. The Lumen global illumination system in Unreal Engine 5.0 [Epic 2021] supports software ray tracing for signed distance fields and hardware accelerated ray tracing for mesh geometry, with some limitations like no support for transparent materials in the former and constraints around complex deforming characters in the latter. The system has no support for cluster distribution of ray tracing for scalability.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SA '21 Technical Communications, December 14–17, 2021, Tokyo, Japan

© 2021 Association for Computing Machinery.

ACM ISBN 978-1-4503-9073-6/21/12...\$15.00

<https://doi.org/10.1145/3478512.3488605>

We choose to build a real time cluster renderer using path tracing not only because of its generality in terms of photorealistic rendering effects but also because it is uniquely suited for massively parallel computing, a quality that has been exploited in the many works related to ray tracing acceleration. Wald et al. [2003] present interactive ray tracing for simple objects without any scattering effects on a cluster of PCs. Benthin et al. [2003] present a global illumination solution using clusters with up to 48 CPUs to achieve interactive frame rates at 2 to 5 fps for static scenes. Jaros et al. [2017] present using MPI to accelerate Blender Cycles on a Xeon Phi-based cluster, and Gerveshi and Looper [2019] present distributed interactive rendering using the Moonray [Lee et al. 2017] renderer; neither system supports real time animation.

While there has been much work done to accelerate and advance ray tracing and path tracing across both offline and real time rendering, we are the first to present a distributed path tracing rendering system that leverages cluster computing and low latency streaming to deliver real time photorealistic rendering of complex scenes and dynamic characters to consumer platforms without special GPU support.

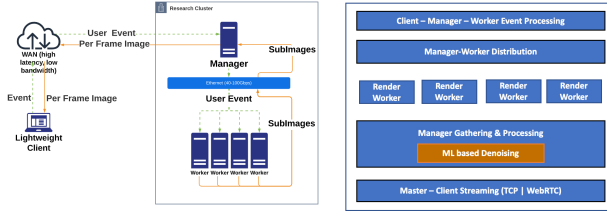


Figure 2: Cluster Rendering Architecture

3 REAL TIME CLUSTER RENDERING SYSTEM

Since our goal is to enable real time path tracing, performance and reliability are the most critical design considerations. The ideal cluster rendering system is one in which the performance and quality of image generation scales linearly with the number of nodes used, with minimum compute and data transfer overhead. We present our cluster rendering architecture designed to achieve these goals.

3.1 Cluster Dataflow Architecture

We choose a single master multiple worker architecture for its simplicity and minimal data transfer overhead. In our system, master serves as the central communication hub and the workload manager. Our design has 3 main benefits:

- It simplified security protection as only the master needs to support public internet service while workers can remain on a private network.
- Latency overhead introduced by the central master forwarding messages to the workers is negligible ($< 1\%$) compared to the network latency and network latency variance between client and master.
- Latency overhead introduced by the central master merging and processing worker results is significantly less than network, compute and synchronization overhead required for the client to directly manage workers.

Figure 2 (left) illustrates the data flow model inside our cluster rendering system, where each client uses a reliable TCP connection to send UserEvents to the master. The most basic UserEvent is the CameraEvent which includes the camera pose information for the next frame. When the master receives the CameraEvent, it immediately forwards it to the workers, each of which will perform a portion of the rendering work, then send the computed results back to the master. The master merges all the worker results and performs any post processing necessary to produce the final rendered image, then sends the final rendered image back to the client for display.

Since intra-cluster network has high bandwidth, low latency and low variance, we use TCP to transmit data between master and workers. For final image delivery from the master to the client, our system supports both TCP based JPEG image streaming and UDP based WebRTC video streaming.

3.2 Render Work Distribution

To achieve linear scalability in performance and quality w.r.t. the number of worker nodes inside a uniform cluster, we want to choose a work distribution strategy with good load balance and minimal overhead in distribution and merging.

Tiling is a common approach to render work distribution where an image of given *width* and *height* is divided into n tiles with fixed tile size. Tiling has small overhead in terms of work distribution and merging cost; its main drawback is that load balance can be uneven depending on the scene variation for small tile counts.

Sample-based distribution leverages the fact that path tracing requires many samples per pixel to converge. By varying the random seed, each worker can generate a different subset of samples per pixel. Each pixel's final radiance is the normalized sum of the radiance computed by the workers. This approach achieves perfect load balance by having each worker compute the same number of samples for each pixel. However, the network bandwidth required to compute the final radiance is linear w.r.t. number of worker nodes because every worker needs to send its full resolution radiance buffer to the master.

Pixel striding divides the rendering work by having each worker render every other pixel. For n number of workers, we use a strategy where each worker renders a different sub-sample of the original image (Figure 3 top). Given an image of size *width* and *height*, and the total number of workers being $n = w_n * h_n$, we ask each worker to generate an image of size $\frac{width}{w_n} * \frac{height}{h_n}$, that are combined together into a final image with $\frac{width}{w_n} * \frac{height}{h_n}$ number of pixel blocks of size (w_n, h_n) .

We map each k th pixel inside the (i, j) pixel block of the final image to the k th worker's radiance value at pixel (i, j) (Figure 3 bottom). This is done by applying a scale $s = (\frac{1}{w_n}, \frac{1}{h_n})$ to the pixel bounds and a translation $t = (s_x \lfloor k/w_n \rfloor, s_y(k \bmod w_n))$ to the pixel center by each k th worker during per pixel camera-ray sample generation.

The bandwidth between the worker and the master is exactly the size of the original radiance buffer and the merge operation for all the workers can be done in parallel since each pixel in each worker's output corresponds to a different pixel in the final image. With pixel

striding, we achieve good load balance with minimal compute and bandwidth overhead in work distribution and merging.

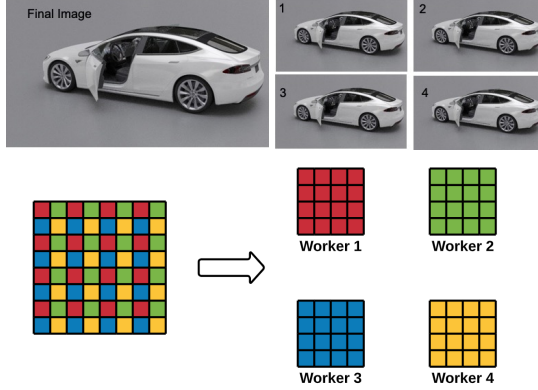


Figure 3: By having each worker render a different sub-sample of the original image, pixel striding delivers near perfect load balance with minimal merging overhead.

3.3 Pipeline Based Parallel Execution

To achieve maximum performance, our cluster rendering system uses pipeline based parallel execution models at the highest level, with task level parallelism and data parallelism in its compute (path tracing) and data (geometry processing) intensive sub-systems where applicable.

Each worker has 2 main threads: a main rendering thread that performs master-forwarded event processing and rendering for frame $n + 1$, and a networking thread that forwards the radiance buffer for frame n to master.

The client also has 2 main threads: a UI thread for processing user events and displaying image for frame $n - 1$, and a networking thread for receiving image (from master) for frame n .

The master has $2 + \text{worker_count}$ main threads:

- (1) A main rendering thread that performs client event processing, work distribution and local rendering for frame $n + 1$.
- (2) One networking thread per worker for receiving each worker's radiance buffer for frame n .
- (3) A post processing thread that performs merging of local and worker radiance buffers; followed by denoising, tone mapping and image streaming for frame n .

Throughout the system, a total of $4 + 3 \times \text{worker_count}$ main threads execute in parallel across 3 pipeline stages. To ensure these threads are running at maximum efficiency without locking or any unnecessary data copy or allocation overhead, fixed sized circular queues are used between each pair of producer and consumer threads. For example, the master's post processing thread is the consumer (reader) of the *local* RadianceBufferQueue produced (rendered) by the main rendering thread (writer); it is also the consumer of the *worker* RadianceBufferQueue produced by the networking thread (writer); as the master is responsible for merging the radiance buffers produced by the workers and its own local rendering to produce the complete radiance buffer.

4 REAL TIME PERSISTENT CYCLES

We choose Cycles (Blender 2.93 version) as the rendering engine for our cluster rendering system because it has most of the rendering features we need in geometry (meshes and curves), shading (image based and programmable), physically based lighting (area lights and environment maps) and BXDF (Disney BSDF, path traced BSSRDF, hair BSDF) models.

Cycles has built-in support for Optix-accelerated path tracing and CUDA accelerated shader evaluation. However, being used primarily as an offline production renderer where each rendered image is treated as completely independent from one another. It lacks some *basic elements* found in real time rendering systems, most notably a persistent scene context with the concept of *frame-time* that can be used to eliminate redoing computation for temporally persistent rendering states.

To enable real time animation rendering, we need to spend our limited compute resources only on states and geometry that *actually* change over time. First, we added the concept of *frame-time* to the *Scene* object and modified Cycles' scene update processing to support *frame-time* notification to the appropriate nodes and procedurals; then we incorporated two experimental features in Blender 3.0, one is support for change registration for Cycles' node graph, and the other is a native Alembic [2015] procedural that enables *efficient* playback of prebaked geometry animation stored in this industry standard format.

Compared with regular Cycles 2.93, these changes culminated in 9x+ speed up in per-frame geometry processing for our digital human character by accelerating animation playback, geometry data transfer (by only transferring vertex data that has changed from previous frame), and BVH compute (by using Optix BVH update which is 10x more efficient than BVH rebuild).

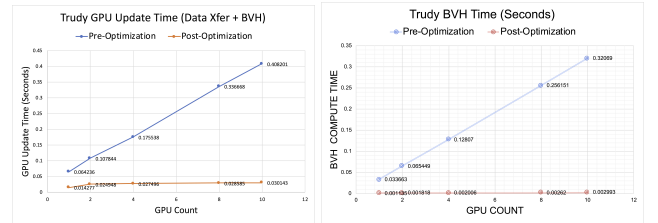


Figure 4: Comparison of multi-GPU geometry computation time pre and post optimization. Left is total GPU geometry update time, right is BVH computation time.

Multiple GPU Performance on a Single Node. Cycles uses tiling for render work distribution on a multi-GPU system, we use around 60 tiles to achieve good scaling on our 10 GPU cluster nodes. Even though each GPU is responsible for a subset of tiles, indirect rays in path tracing require fast random access to the entire scene and rendering context (including the BVH). High data transfer cost between GPUs means that the most efficient approach is to mirror all scene changes, including per frame vertex data changes and BVH computation. We replace Cycles' built-in serialized data transfer and parallel BVH compute with efficient parallel data transfer and parallel BVH compute. Figure 4 shows our optimization changed the multi-GPU scaling curve from linear to (near) constant, and delivered 13x (.40s to 0.03s) speed up in total geometry processing

(data transfer + BVH) and 100x speed up (0.32s to 0.003s) in BVH computation for the Trudy scene.

5 RESULTS

We present testing results on a 32 node cluster connected with 40Gb Ethernet¹. Each node is equipped with 2 Xeon Gold 6138 (2.00GHz) and 10 Quadro RTX 8000 (48GB PCIe x16). Every worker and master node in our tests uses all 10 GPUs; therefore, $total_gpu_used = 10 * nodes_used$.

We use two representative scenes that highlight different rendering effects. Tesla uses glossy reflection and refraction, Trudy uses path traced subsurface scattering and physically based eye shading; both test scenes support inter-object reflections and occlusions with maximum ray depth set to 10.

Table 1 (left) shows that when total samples per pixel is fixed for Tesla (fixed quality), increasing the number of working nodes linearly increases frame rate. Table 1 (right) shows that when frame rate is fixed for Trudy (fixed time), increasing number of working nodes linearly improves rendering quality (total samples per pixel increases from 50 to 100)².

Detailed timing breakdown reported in Table 1 shows that our system throughput is limited by per frame scene update and render time with $fps \approx \frac{0.9}{update_time + render_time}$. The cluster related overhead of our system (master distribution + merge time) is not only significantly less than rendering time but also completely hidden by our pipeline based parallel execution model w.r.t. their impact on rendering throughput (final frame rate).

By making design choices that minimize work distribution overhead and maximize parallel execution efficiency, we have built the first real-time cluster path tracing renderer that can scale linearly in performance and quality for up to 100 GPUs.

Table 1: Performance measurements for real-time rendering of Tesla using 2 and 5 cluster nodes, and of Trudy using 5 and 10 cluster nodes. Worker time reported is the average time for all the workers.

Scene	Tesla		Trudy	
Working Nodes	2	5	5	10
Working GPUs	20	50	50	100
Per Node SPP	15	6	10	10
Total SPP	30	30	50	100
Client FPS	15	27	14	14
Timing Breakdown	ms			
Master render	60.9	32.6	29.5	29.9
Worker render	59.1	26.4	34.0	39.5
Master scene update	1.2	1.0	33.2	33.4
Worker scene update	0.8	0.8	31.2	29.2
Master update + render	62.1	33.6	62.7	64.3
Worker update + render	59.9	27.2	65.2	68.7
Master tone mapping	2.0	2	1.6	1.6
Master compression	8.9	8.6	11.5	9.3
Master denoising	14.7	13.9	0	0

¹Node-node latency $\approx 0.1ms$

²Rendering quality measured as variance reduction improves as square root of sample count

6 CONCLUSION

We presented the design and implementation of the first production quality real time cluster path tracing rendering system. Our system applies near-optimal work distribution and pipeline based parallel execution models to deliver almost perfect scaling in path tracing quality and performance in a cluster of RTX nodes connected with high bandwidth interconnect. However, there remain many optimization and quality improvement opportunities throughout the system and its core rendering engine that we plan to explore in the future.

ACKNOWLEDGMENTS

Thanks to Brecht Van Lommel and Kevin Dietrich for their work on the Cycles Alembic Procedural. Thanks to Fang Yu, Brian Budge, Mark Segal, and David Blythe for their comments and suggestions on the paper revisions.

REFERENCES

- Carsten Benthin, Ingo Wald, and Philipp Slusallek. 2003. A Scalable Approach to Interactive Global Illumination. *Computer Graphics Forum* (2003). <https://doi.org/10.1111/1467-8659.t01-2-00710>
- Blender. 2021. Cycles Open Source Production Rendering. <http://www.cycles-renderer.org/>
- Brent Burley, David Adler, Matt Jen-Yuan Chiang, Hank Driskill, Ralf Habel, Patrick Kelly, Peter Kutz, Yining Karl Li, and Daniel Teece. 2018. The Design and Evolution of Disney's Hyperion Renderer. *ACM Trans. Graph.* 37, 3, Article 33 (July 2018), 22 pages. <https://doi.org/10.1145/3182159>
- Per Christensen, Julian Fong, Jonathan Shade, Wayne Wooten, Brenden Schubert, Andrew Kensler, Stephen Friedman, Charlie Kilpatrick, Cliff Ramshaw, Marc Bannister, Brenton Rayner, Jonathan Brouillat, and Max Liani. 2018. RenderMan: An Advanced Path-Tracing Architecture for Movie Rendering. *ACM Trans. Graph.* 37, 3, Article 30 (Aug. 2018), 21 pages. <https://doi.org/10.1145/3182162>
- Epic. 2021. Lumen Global Illumination and Reflection System. <https://docs.unrealengine.com/5.0/en-US/RenderingFeatures/Lumen/TechOverview/>
- Luca Fascione, Johannes Hanika, Mark Leone, Marc Droske, Jorge Schwarzhaupt, Tomáš Davidovič, Andrea Weidlich, and Johannes Meng. 2018. Manuka: A Batch-Shading Architecture for Spectral Path Tracing in Movie Production. *ACM Trans. Graph.* 37, 3, Article 31 (Aug. 2018), 18 pages. <https://doi.org/10.1145/3182161>
- Iliyan Georgiev, Thiago Ize, Mike Farnsworth, Ramón Montoya-Vozmediano, Alan King, Brecht Van Lommel, Angel Jimenez, Oscar Anson, Shinji Ogaki, Eric Johnston, Adrien Herubel, Declan Russell, Frédéric Servant, and Marcos Fajardo. 2018. Arnold: A Brute-Force Production Path Tracer. *ACM Trans. Graph.* 37, 3, Article 32 (Aug. 2018), 12 pages. <https://doi.org/10.1145/3182160>
- Alex Gerveshi and Sean Looper. 2019. Distributed Multi-Context Interactive Rendering. In *Proceedings of the 2019 Digital Production Symposium* (Los Angeles, California) (DigiPro '19). Association for Computing Machinery, New York, NY, USA, Article 4, 3 pages. <https://doi.org/10.1145/3329715.3338878>
- Sony Pictures Imageworks and Lucasfilm Ltd. 2015. Alembic. <https://www.alembic.io/>
- Milan Jaros, Lubomir Riha, Tomas Karasek, Petr Strakos, and Daniel Krpelik. 2017. Rendering in Blender Cycles Using MPI and Intel® Xeon Phi™. In *Proceedings of the 2017 International Conference on Computer Graphics and Digital Image Processing* (Prague, Czech Republic) (CGDIP '17). Association for Computing Machinery, New York, NY, USA, Article 2, 5 pages. <https://doi.org/10.1145/3110224.3110236>
- Christopher Kulla, Alejandro Conty, Clifford Stein, and Larry Gritz. 2018. Sony Pictures Imageworks Arnold. *ACM Trans. Graph.* 37, 3, Article 29 (Aug. 2018), 18 pages. <https://doi.org/10.1145/3180495>
- Mark Lee, Brian Green, Feng Xie, and Eric Tabellion. 2017. Vectorized Production Path Tracing. In *Proceedings of High Performance Graphics* (Los Angeles, California) (HPG '17). Association for Computing Machinery, New York, NY, USA, Article 10, 11 pages. <https://doi.org/10.1145/3105762.3105768>
- Steven Parker, William Martin, Peter-Pike J. Sloan, Peter Shirley, Brian Smits, and Charles Hansen. 1999. Interactive Ray Tracing. In *Proceedings of the 1999 Symposium on Interactive 3D Graphics* (Atlanta, Georgia, USA) (I3D '99). Association for Computing Machinery, New York, NY, USA, 119–126. <https://doi.org/10.1145/300523.300537>
- I. Wald, C. Benthin, and P. Slusallek. 2003. Distributed interactive ray tracing of dynamic scenes. In *IEEE Symposium on Parallel and Large-Data Visualization and Graphics*, 2003. *PVG 2003*, 77–85. <https://doi.org/10.1109/PVGS.2003.1249045>